



جلسه فنی تیم ۶۰۰ دقیقه

برنامه نویسی عامل محور Agentic Coding

از نیازمندی کسب و کار تا تست، با Rule، Spec، و Skill

Cursor · Claude Code · GitHub Copilot · Codex · Kiro · Spec Kit

دستور جلسه

۱. چرا الان؟

داده‌ها، تکامل ابزارها، پارادوکس بهره‌وری

۲. مفاهیم پایه

LLM، Agent، Context، Harness، Spec / Rule / Skill

۳. توسعه مبتنی بر Spec

دیدگاه Martin Fowler / Birgitta Böckeler و نحوه نوشتن

۴. تیم از A تا Z

کسب‌وکار ← Tester ← Developer ← Tech Lead ← PO

۵. مقایسه ابزارها

کدام ابزار برای کدام کار، ساختار پوشه‌ها

۶. عمل و پرسش‌ها

نحوه صحبت با عامل، ضدالگوها، نقشه راه، FAQ

مطالعه کنترل‌شده METR

-19%

توسعه‌دهندگان باتجربه با AI **کندتر** شدند، در حالی که فکر می‌کردند ۲۰٪ سریع‌ترند.

تسک‌های واقعی در ریپوهای بزرگ متن‌باز

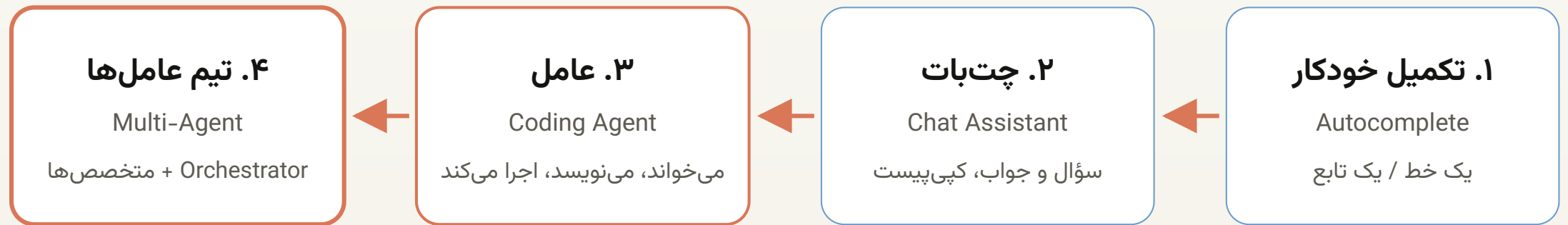
گزارش DORA 2025 (گوگل)

90%

از توسعه‌دهندگان از AI استفاده می‌کنند؛ بیش از ۸۰٪ احساس بهره‌وری بیشتر دارند.

سرعت تحویل ↑ اما پایداری تحویل ↓

«AI یک تقویت‌کننده است: نقاط قوت و ضعف سازمان را بزرگ‌تر می‌کند.»، DORA 2025



نقش انسان: از «تایپ کننده» به «طراح و ناظر حلقه»

02

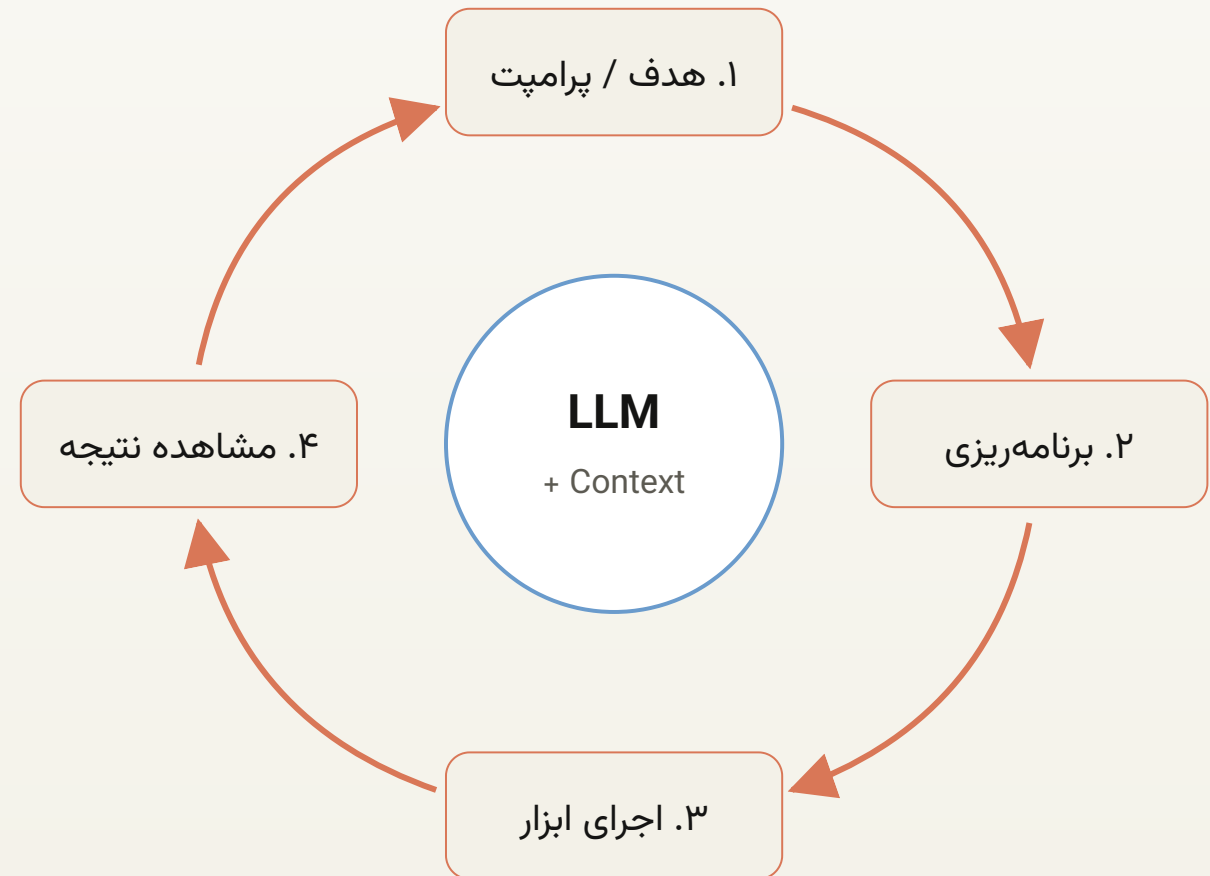
مفاهیم پایه

LLM، Agent، Context Engineering، Harness و چهار artifact اصلی

LLM، دستیار کدنویسی، عامل، تفاوت چیست؟

چيست؟	LLM (مدل زبانی)	دستیار کدنویسی AI	عامل کدنویسی (Agent)
مثال	Claude, GPT, Gemini	Copilot completions, Chat	Cursor Agent, Claude Code, Codex, Kiro
دسترسی	فقط متن ورودی	فایل باز / انتخاب شده	کل ریپو، ترمینال، تست، مرورگر، MCP
چه کسی اجرا می‌کند؟	شما	شما (کپی/قبول)	خود عامل؛ شما بازبینی می‌کنید
بهترین برای	ایده، توضیح، تحلیل	تکمیل سریع، سؤال موضعی	فیچر چندفایلی، رفع باگ، refactor، تست

Agent = Model + Harness - یعنی «مدل» فقط بخشی از ماجراست.



- ابزارها: خواندن/جستجوی فایل، ویرایش، ترمینال، تست، مرورگر، MCP
- مشاهده: خروجی build، خطای تست، lint
- حلقه تا رسیدن به هدف یا توقف تکرار می‌شود
- بدون معیار «موفقیت» قابل اندازه‌گیری، عامل خودش به خودش نمره می‌دهد!

«Context engineering یعنی گزینش آنچه مدل می‌بیند، تا نتیجه بهتری بگیرید.»

- Bharani Subramaniam، نقل‌شده در مقاله Birgitta Böckeler، martinowler.com (فوریه ۲۰۲۶)

Specs

قرارداد فیچر فعلی

Skills

رویه‌های قابل بارگذاری در صورت نیاز

Rules

دستورالعمل‌های دائمی پروژه

خود کدبیس

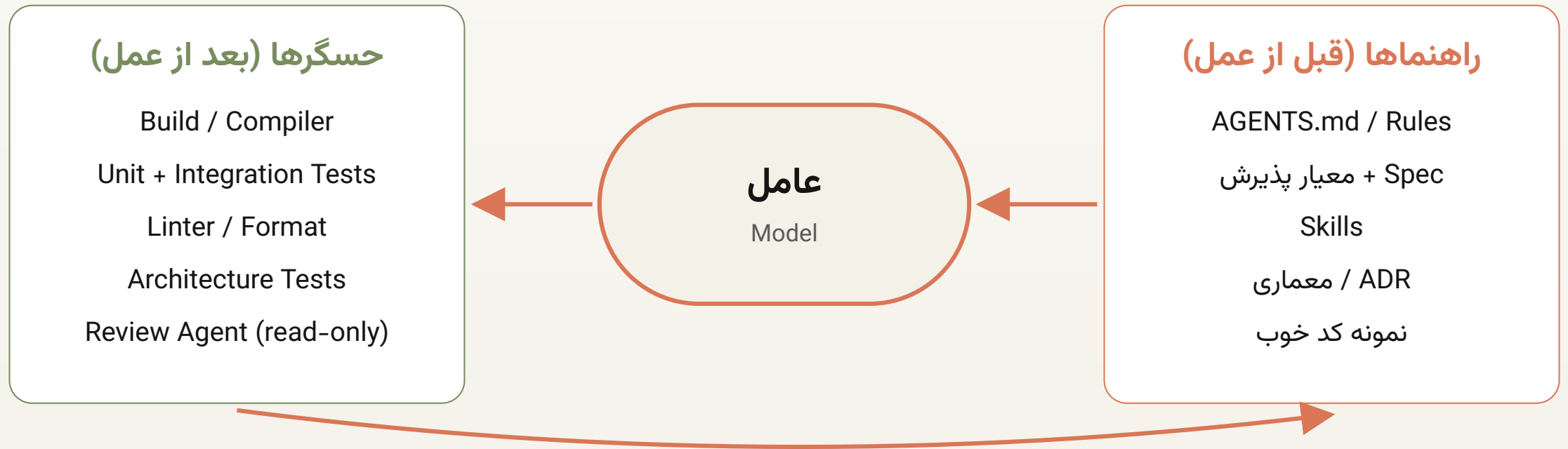
کد تمیز = بهترین context

Subagents

context جدا برای تحقیق و بازبینی

MCP

اتصال به DB، Jira، مستندات، مرورگر



بازخورد: اگر نتیجه بد بود، Harness را اصلاح کن، نه فقط خروجی را

چهار artifact که نباید قاطی شوند

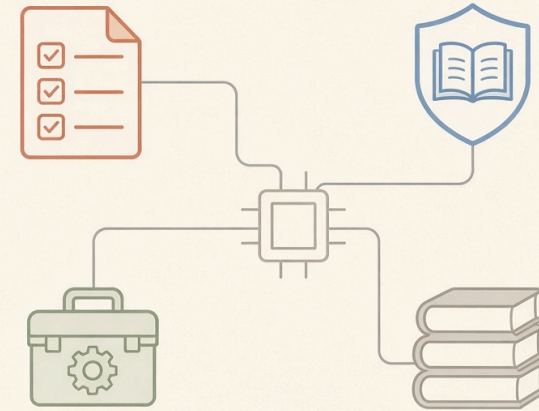
= این بار چه بسازیم Spec

= همیشه اینجا چطور کار می‌کنیم Rule

= این نوع کار را چطور انجام بدهیم Skill

= کسب‌وکار چه معنایی دارد Business Doc

اشتباه شماره ۱ تیم‌ها: کپی کردن کل BRD داخل فایل Rule.



مثال جمله Spec

«Given قسطی با بیش از ۳۰ روز تأخیر، When لیست پیگیری درخواست شود،
Then آن قسط با برچسب Overdue نمایش داده شود.»
قابل تست ✓ ۰ مختص این فیچر ✓

توصیف یک فیچر یا تغییر مشخص: هدف، داستان کاربر، معیار پذیرش،
خارج از محدوده، لبه‌ها.

- عمر: برای یک فیچر/تسک (در حالت spec-anchored نگه داشته می‌شود)
- کی بارگذاری می‌شود: فقط وقتی روی همان فیچر کار می‌شود
- نویسنده: PO + Tech Lead
- مصرف‌کننده: عامل توسعه، Developer، Tester
- محل: specs/001-feature/spec.md

مثال Rule

- Domain must not reference Infrastructure.
- Money fields use decimal, never double.
- Never hardcode secrets.
- Run: dotnet build && dotnet test

دائمی ✓ · کوتاه و دستوری ✓

سیاست **دائمی** پروژه که عامل در تقریباً هر ویرایش مرتبط باید رعایت کند.

- مرزهای معماری (Clean Architecture)
- نام‌گذاری و ساختار پوشه
- حداقل‌های تست، امنیت، دستورهای build
- محل: AGENTS.md، .cursor/rules/*.mdc، CLAUDE.md، .github/copilot-instructions.md
- مالک: Tech Lead، مثل کد review شود

۱. کشف

name + description (~100 توکن)

۲. فعال سازی

کل SKILL.md (> ۵۰۰۰ توکن)

۳. اجرا

references / scripts فقط در صورت نیاز

Progressive Disclosure

یک رویه قابل استفاده مجدد که عامل فقط وقتی لازم است بارگذاری می کند. استاندارد باز (اصالتاً از Anthropic) که Claude Code، Copilot و Cursor، Codex پشتیبانی می کنند.

```
.agents/skills/dotnet-cqrs-feature/
  SKILL.md           # name + description + steps
  references/         # optional docs
  scripts/           # optional helpers
  assets/            # optional templates
```

قانون کسب و کار: «اقساط معوق بیش از ۳۰ روز باید پیگیری شوند»

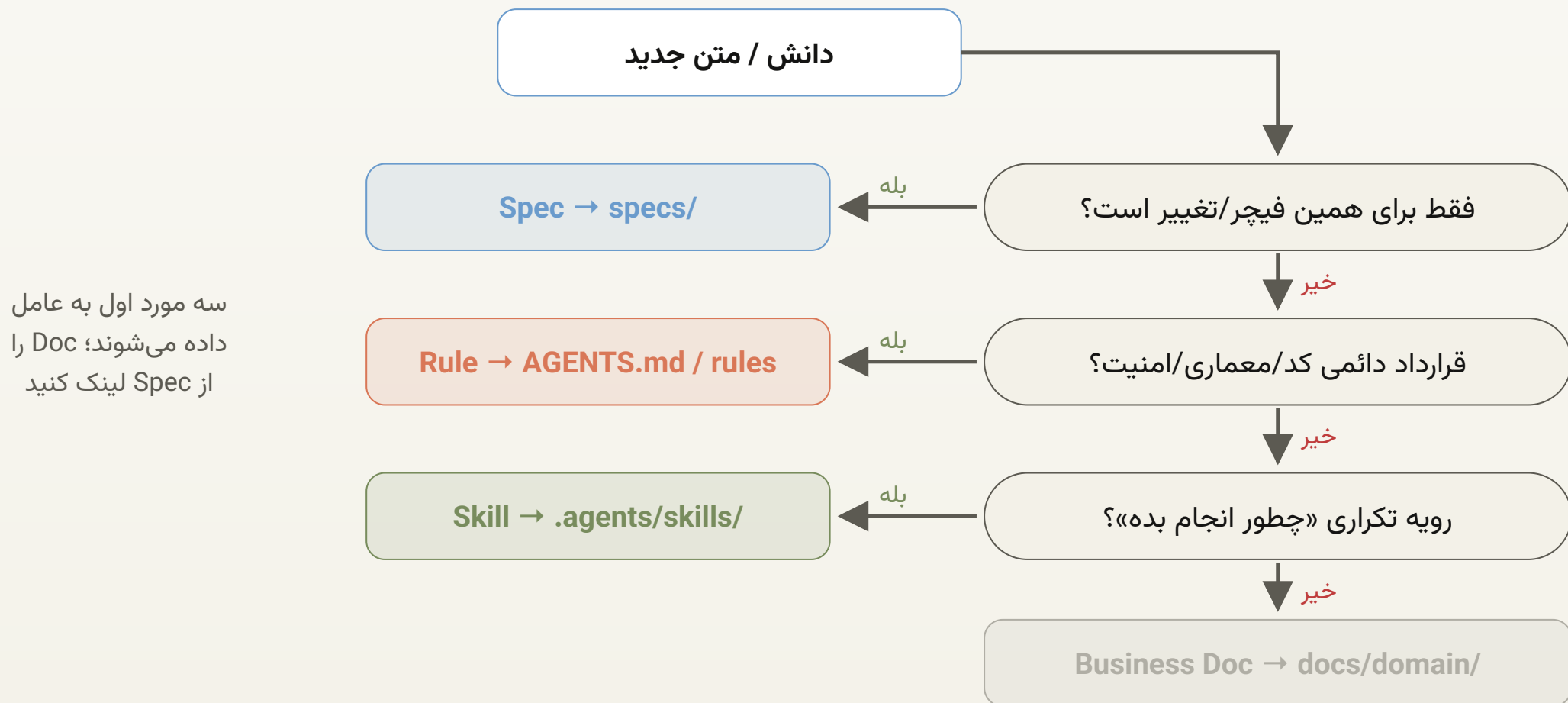
- ✓ در مستند دامنه (حقیقت برای انسان)
- ✓ در Spec فیچر مربوطه، به شکل معیار پذیرش
- ✓ اختیاری: در references یک Skill اگر مکرر لازم است
- ✗ در cursor/rules، مگر قرارداد دائمی کدنویسی باشد

مستندات کسب و کار = دانش دامنه برای انسان‌ها (و گاهی عامل):
واژه‌نامه، روایت فرایند، BRD، نقشه فرایند.

به طور خودکار بارگذاری نمی‌شود؛ از Spec به آن لینک بدهید یا خلاصه‌اش را در Skill/references بگذارید.

محل پیشنهادی: docs/domain/glossary.md،
docs/domain/installments.md

درخت تصمیم: این متن را کجا بگذارم؟



Rules در برابر Skills

Skills	Rules	
آموزش یک قابلیت یا گردش کار تخصصی	استاندارد و قرارداد کدنویسی	هدف
در صورت نیاز (تدریجی)	همیشه یا با glob (کل متن)	بارگذاری
دستورالعمل + اسکریپت + مثال + منابع	فقط دستورالعمل	محتوا
می تواند بلندتر باشد	کوتاه (هزینه context دائمی)	طول مناسب
استاندارد باز واحد (agentskills.io)	فرمت های مختلف به ازای هر ابزار	قابلیت حمل
«مراحل افزودن فیچر CQRS» / «ساخت migration»	«Domain به Infrastructure وابسته نباشد»	مثال
اگر Skill همیشه لازم است و کوتاه ← Rule کن	اگر Rule شبیه آموزش گام به گام شد ← Skill کن	سیگنال جابه جایی

Hook و MCP، Subagent

Hook

اسکریپت قطعی در رویدادهای عامل (قبل/بعد ابزار، پایان).

«Rule توصیه است؛ Hook تضمین است.»

Subagent

عامل کمکی با **context جدا** و ابزار محدود؛ فقط خلاصه برمی‌گرداند.

تحقیق کدبیس، بازبینی read-only، تست

MCP

Model Context Protocol، پروتکل استاندارد اتصال عامل به سیستم‌های بیرونی.

Jira، GitLab، دیتابیس، مستندات، مرورگر

مثال: بعد از هر ویرایش فایل cs. ← Hook اجرای dotnet build؛ قبل از subagent ← commit بازبین.

03

توسعه مبتنی بر Spec

Spec-Driven Development، دیدگاه martinfowler.com و نحوه نوشتن Rule، Spec و Skill

چرا Spec؟ Vibe Coding در برابر کار ساختاریافته

Spec-Driven

- نیازمندی ← Tasks ← Plan ← Spec ← کد
- همراستایی PO و فنی قبل از مصرف توکن
- معیار پذیرش = حسگر عامل
- Spec در ریپو می ماند و نسخه بندی می شود
- بازبینی در برابر Spec، نه حافظه

Vibe Coding

- پرامپت ← کد ← امید!
- عامل «چیز اشتباهی» را درست می سازد
- هیچ معیار پذیرشی برای بازبینی نیست
- دانش در تاریخچه چت گم می شود
- برای نمونه اولیه خوب؛ برای محصول پرریسک

DX / Laura Tacho: مشکل اصلی تیم ها «همراستایی» است، SDD تلاشی برای حل همین است.

۱. Spec-first

Spec خوب اول نوشته می‌شود و برای همین تسک استفاده می‌شود؛ بعد از تسک ممکن است کنار گذاشته شود.

۲. Spec-anchored

Spec بعد از اتمام تسک نگه داشته می‌شود و برای تکامل و نگهداری همان فیچر استفاده می‌شود.

۳. Spec-as-source

فقط Spec توسط انسان ویرایش می‌شود؛ کد تولیدی است و دستی ویرایش نمی‌شود. (هنوز آزمایشی)



SDD چطور به تیم ما کمک می‌کند؟

کاهش دوباره‌کاری

کمتر «عامل چیز اشتباه ساخت»

دروازه کیفیت

معیار پذیرش ← تست ← حسگر عامل

هم‌راستایی

PO و Tech Lead قبل از کد توافق می‌کنند

رشد جونیورها

Spec = بریف، Skill = روش انجام

بازبینی بهتر

Reviewer در برابر Spec چک می‌کند

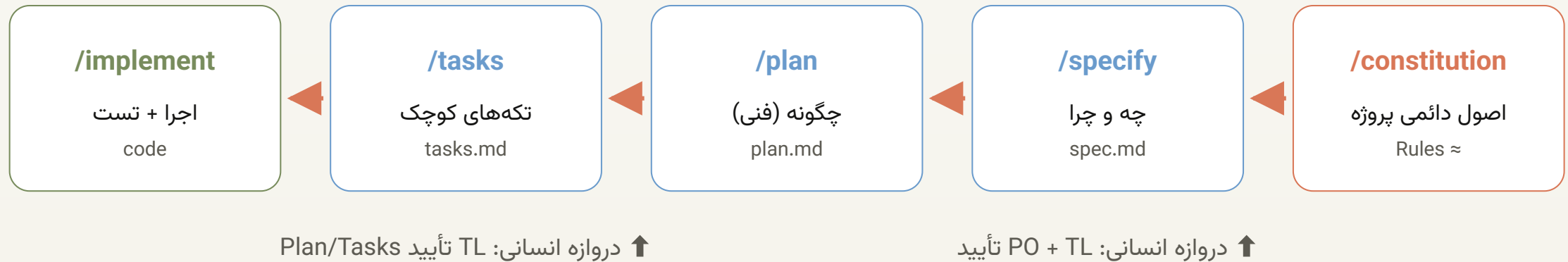
تحویل شفاف

Tasks ← Plan ← Spec ← BRD ← کد ← تست

سه ابزار SDD کنار هم

	Kiro (AWS)	Spec Kit (GitHub)	Tessl
نوع	IDE عامل محور	CLI + slash command در ابزار شما	فریم ورک + رجیستری Skill
جریان	Requirements ← Design ← Tasks	Constitution ← Specify ← Plan ← Tasks ← Implement	Spec ← کد تولیدی
سطح (Böckeler)	سبک، spec-first	قابل سفارشی ترین، عملاً spec-first	تنها هدف گذاری -as / spec-anchored source
ابزارهای پشتیبان	خود Kiro	~۳۰ عامل: Copilot، Claude، Cursor، ...	Claude Code، Codex، Gemini، ...
مناسب برای	تیم های AWS، شروع سریع	تیم هایی که artifact در ریپو می خواهند	آزمایش آینده نگر

نقد رایج کاربران: «Spec Kit گاهی شبیه آبشار (waterfall) برای عامل هاست»، Spec را سبک نگه دارید.



نمونه کامل در ریپو: `samples/specs/001-installment-followup/` (spec.md, plan.md, tasks.md)

چک لیست کیفیت

- ☐ هر معیار با تست یا دمو قابل اثبات است
- ☐ معماری مفصل اینجا نیست (برود در Plan / ADR)
- ☐ اندازه یک PR یا اپیک کوتاه
- ☐ واژه‌ها هم‌خوان با واژه‌نامه کسب‌وکار
- ☐ Non-goals روشن (جلوی scope creep عامل)
- ☐ بعد از merge نگه داشته و به‌روز شود

قالب حداقلی

1. هدف (یک پاراگراف)
2. داستان‌های کاربر
3. معیار پذیرش قابل تست (Given/When/Then)
4. خارج از محدوده (Non-goals)
5. قوانین دامنه مرتبط (لینک به docs)
6. لبه‌ها و خطاها
7. وابستگی‌ها
8. Definition of Done

نمونه واقعی Spec، پیگیری اقساط معوق

هدف: فروشنده اقساط معوق را شناسایی و اقدام پیگیری ثبت کند تا وضعیت شفاف شود.

معیار پذیرش:

1. Given قسط با بیش از ۳۰ روز تأخیر، When لیست درخواست شود، Then با برچسب Overdue بیاید.
2. Given قسط معوق، When پیگیری ثبت شود، Then رکورد با CreatedAt و CreatedBy ذخیره شود.
3. Given پیگیری‌ها، When جزئیات خوانده شود، Then تاریخچه جدید به قدیم برگردد.
4. Given ورودی نامعتبر، When ثبت شود، Then خطای اعتبارسنجی/NotFound و هیچ ذخیره‌ای.

خارج از محدوده: ارسال خودکار پیامک، محاسبه جریمه، ال وب.

قانون دامنه (از docs): هر اقدام پیگیری باید قابل ممیزی باشد.

چگونه خوب بنویسیم Rule

.cursor/rules/dotnet-clean-architecture.mdc

```
---
description: Clean Architecture boundaries
globs: "**/*.{cs,csproj}"
alwaysApply: false
---
# .NET Clean Architecture
## Do
- Depend inward: Host → Application → Domain
- Business rules in Domain/Application
- Use Result for expected failures
## Don't
- Reference Infrastructure from Domain
- Use DbContext in controllers
- Invent folder layouts without an ADR
```

• ✓ خوب:

- کوتاه، دستوری: Do / Don't
- دائمی، نه مخصوص یک تیکت
- با globs محدود شود
- چند فایل کوچک بهتر از یک فایل غول

• ✗ بد:

- مقاله بلند / آموزش گام به گام (← Skill)
- دستور مخصوص یک فیچر (← Spec)
- کپی (← docs) BRD

چگونه Skill خوب بنویسیم

`.agents/skills/dotnet-cqrs-feature/SKILL.md`

```
---
name: dotnet-cqrs-feature
description: Add a MediatR CQRS feature (Command/Query,
  Handler, Validator, Controller). Use when the user asks
  to add a new API feature, use-case, command, or query.
---
## Steps
1. Read specs/**/spec.md (acceptance criteria)
2. Application: Command/Query + Handler + Validator
3. Infrastructure: repository methods if missing
4. Host: thin controller → MediatR
5. Tests: happy path + validation failure
6. Verify: dotnet build && dotnet test
## Output checklist
- [ ] No business logic in controller
- [ ] Spec acceptance criteria covered
```

- name: کوچک، با خط تیره، هم نام پوشه
- description: چه می‌کند + کی فعال شود (تمام بار trigger)
- بدنه: مراحل شماره‌دار، دستور دقیق، چک لیست خروجی
- زیر ۵۰۰۰ توکن؛ اضافه‌ها در references/
- با یک کار واقعی تیم تست شود
- رویه تکراری باشد، نه فیچر یک‌باره

دامها و نقدها

⚠️ آبخار برای عاملها

Spec سنگین و فرمهای زیاد = از دست رفتن چابکی. سبک شروع کنید.

⚠️ اصطلاحات هنوز سیال است

Böckeler: حتی درک مشترکی از «Spec» و تفاوتش با instruction file وجود ندارد.

⚠️ Rules داخل BRD

context شلوغ، قوانین مبهم، هزینه توکن بالا، کیفیت پایین.

⚠️ رانش Spec (Drift)

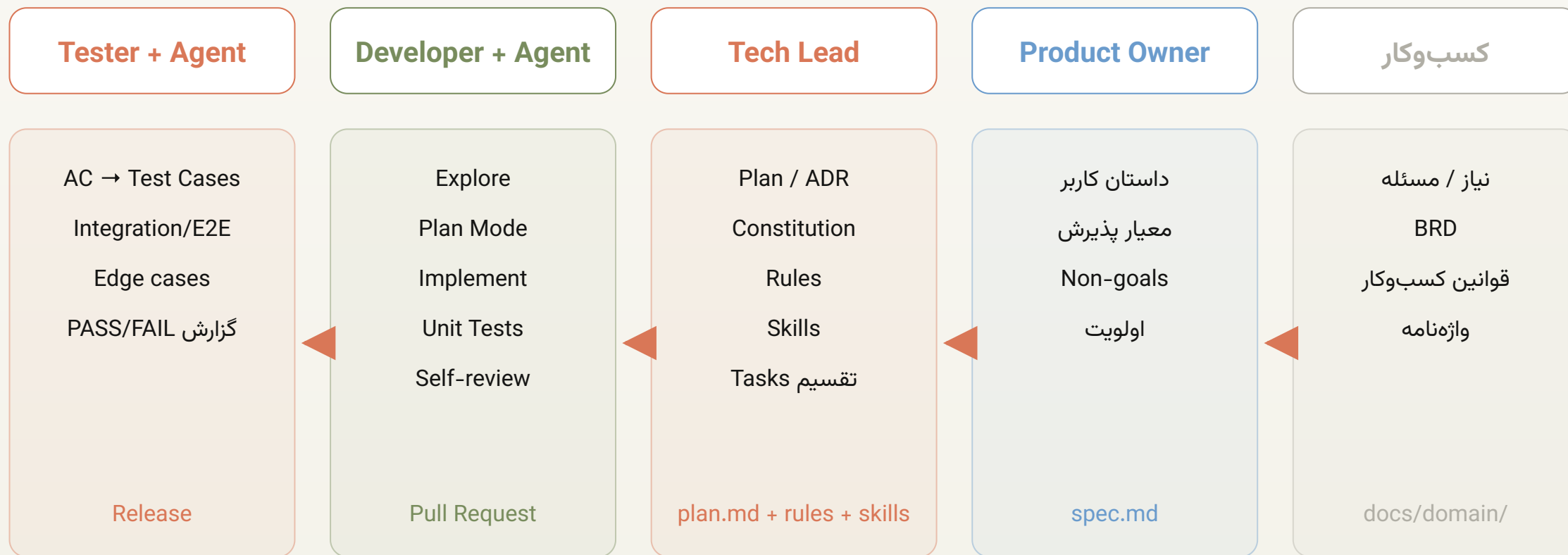
Spec-first بدون بهروزرسانی ← Spec دروغ می‌گوید. فیچرهای مهم را anchored کنید.

04

تیم از A تا Z

کسب و کار ← Tester ← Developer ← Tech Lead ← PO، هر نقش با عامل‌ها

جریان کامل: از نیازمندی تا تست



P0 و Tech Lead با عامل‌ها چه می‌کنند؟

Tech Lead

- مالک Rules، Constitution، AGENTS.md
- ساخت و نگهداری Skills تیم
- تأیید Plan و تقسیم Tasks
- طراحی Harness: تست معماری، CI، Hook
- تعیین سطح خودمختاری عامل برای هر نوع کار
- خروجی: plan.md + rules + skills

Product Owner

- تبدیل BRD به داستان کاربر با کمک عامل
- درخواست از عامل: «معیار پذیرش Given/When/Then و لبه‌ها را پیشنهاد بده»
- اتصال MCP به Jira: ساخت تیکت از Spec
- بررسی Spec از نظر ابهام و تناقض
- خروجی: spec.md تأییدشده

Developer و Tester با عامل‌ها چه می‌کنند؟

Tester / QA

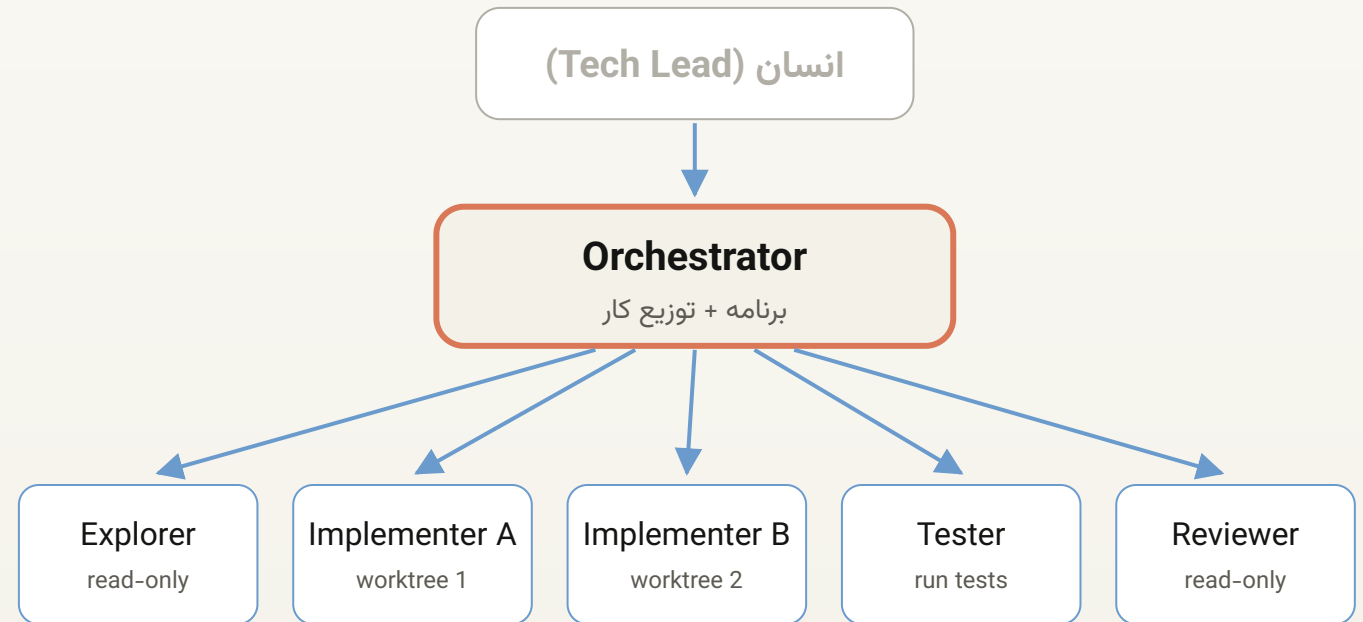
- تبدیل معیار پذیرش Spec به test case
- تولید تست integration / E2E با عامل
- درخواست: «لبه‌هایی که Spec پوشش نداده را پیدا کن»
- عامل مرورگر برای تست UI
- گزارش شکاف Spec به PO
- خروجی: PASS/FAIL در برابر AC

Developer

- **Explore:** «با subagent بررسی کن follow-up فعلی چطور کار می‌کند»
- **Plan:** Plan Mode، فایل‌ها و ترتیب را تأیید کن
- **Implement:** با dotnet-cqrs-feature Skill
- **Verify:** build + test تا سبز شدن
- **Review:** subagent بازبین read-only قبل از PR
- مالک نهایی کد = انسان

الگوی ارکستراسیون عامل‌ها

- Orchestrator خودش کد نمی‌نویسد؛ تقسیم می‌کند
- هر متخصص context و ابزار محدود دارد
- کار موازی در **git worktree** جدا
- Explorer و Reviewer فقط خواندنی
- حداکثر ۳ دور اصلاح، بعد ارجاع به انسان



دروازه‌های انسانی: تأیید Plan · تأیید Merge
PR عامل = همان CI و branch protection مثل PR انسان

✓ On the Loop

انسان **Harness** را می‌سازد و بهبود می‌دهد: Spec،
تست، Rule، Skill، Pipeline.
مقیاس‌پذیر

In the Loop

انسان هر خط را بازبینی و دستی اصلاح می‌کند.
گلوگاه می‌شود

Vibe Coding

انسان بیرون از حلقه؛ هر چه عامل ساخت قبول.
برای نمونه‌سازی

وقتی از خروجی راضی نیستید: «درون حلقه» = خروجی را اصلاح کن. «روی حلقه» = چیزی که آن خروجی را تولید کرد اصلاح کن.

دروازه تأیید و سطح خودمختاری

نوع کار	خودمختاری
مستندات، تست نویسی ماژول پوشش دار	بالا
ارتقای وابستگی با تست کامل	بالا
فیچر معمولی با Spec	متوسط (تأیید Plan)
احراز هویت، پرداخت، بیمه نامه	پایین (پیشنهاد فقط)
Migration داده	پایین + تأیید صریح

دروازه «انجام شد» (Verification Gate)

1. تست های جدید/مرتبط پاس
 2. کل suite پاس (بدون regression)
 3. Build و lint تمیز
 4. معیارهای پذیرش Spec پوشش داده شده
 5. برای تغییر پرریسک: تأیید انسان
- «ارزیابی خود مدل از اتمام، کم اعتمادترین سیگنال است.»

05

مقایسه ابزارها

Cursor · Claude Code · GitHub Copilot · Codex · Kiro · Spec Kit

ماتریس ابزارها (۲۰۲۶)

ابزار	رابط	مدل‌ها	نقطه قوت	مناسب برای
Cursor	IDE کامل + عامل پس‌زمینه	چند مدل	ویرایش تعاملی، Rules با subagent، glob	کار روزانه داخل IDE
Claude Code	ترمینال + IDE	Anthropic	استدلال عمیق، refactor بزرگ، Skill، Hook	کارهای چندفایلی سخت
GitHub Copilot	افزونه IDE + GitHub + CLI	چند مدل	یکپارچگی GitHub، Issue→PR، code review	سازمان‌های GitHub-محور
OpenAI Codex	CLI + ابر	OpenAI	تسک ناهمگام در sandbox ابری	«بفرست و فراموش کن»
Kiro	IDE	چند مدل	Spec-first داخلی	تیم‌های AWS، شروع SDD
Spec Kit	CLI روی ابزار شما	-	قالب SDD برای ~۳۰ عامل	افزودن SDD به ابزار فعلی

بسیاری از تیم‌ها دو ابزار را ترکیب می‌کنند: یکی برای ویرایش روزانه، یکی برای کارهای سنگین واگذارشده.

هر ابزار Rule و Skill را از کجا می‌خواند؟

ابزار	Rules / Instructions	Skills	Subagents
Cursor	.cursor/rules/*.mdc, AGENTS.md, CLAUDE.md	.agents/skills, .cursor/skills	✓
Claude Code	CLAUDE.md (+ @AGENTS.md import)	.claude/skills	.claude/agents/*.md
Copilot	.github/copilot-instructions.md, .github/instructions/*.instructions.md, AGENTS.md	.github/skills, .agents/skills	(agent mode) ✓
Codex	AGENTS.md (nested)	.agents/skills	✓
Kiro	.kiro/steering/*.md	-	-

AGENTS.md = استاندارد بی‌طرف (بنیاد ۶۰، Linux هزار+ ریپو) · نزدیک‌ترین فایل به کد ویرایش‌شده برنده است.

ساختار پوشه پیشنهادی (همه ابزارها با هم)

- ریشه + `.cursor/rules` **Rule**
- `.agents/skills` **Skill**
- `specs/` **Spec**
- `docs/` **Doc**
- همه در git، همه review می‌شوند
- نمونه کامل در پوشه `samples/` این ریپو

```
repo/
├── AGENTS.md                # single source of truth (rules)
├── CLAUDE.md                # "@AGENTS.md" + Claude-only notes
├── .github/
│   └── copilot-instructions.md # short, points to AGENTS.md
├── .cursor/rules/
│   ├── dotnet-clean-architecture.mdc # globs: **/*.cs
│   └── testing.mdc                  # globs: **/*Tests*/**
├── .agents/skills/          # cross-tool skills standard
│   ├── dotnet-cqrs-feature/SKILL.md
│   └── ef-migration/SKILL.md
├── .claude/agents/
│   └── code-reviewer.md       # read-only subagent
├── specs/
│   └── 001-installment-followup/
│       ├── spec.md
│       ├── plan.md
│       └── tasks.md
├── docs/
│   ├── domain/glossary.md    # business knowledge
│   └── adr/                  # architecture decisions
└── src/ tests/
```

کدام ابزار برای کدام کار؟

ویرایش روزانه، فیچر کوچک

Cursor Agent / Copilot Agent mode

سؤال سریع، توضیح کد

چت با LLM یا Ask mode در IDE

کار موازی ناهمگام

Codex cloud / Cursor background agents / Copilot coding agent

Refactor بزرگ، باگ پیچیده

Plan Mode + subagent با Claude Code / Cursor

بازبینی PR

Copilot code review / Bugbot / subagent read-only

فیچر جدید با ذی‌نفعان متعدد

Kiro (SDD) یا Spec Kit

مهم‌تر از ابزار: Harness شما (Rules، Specs، Skills)، این‌ها بین ابزارها قابل انتقال‌اند.

06

عمل و پرسش‌ها

چطور با عامل صحبت کنیم، ضدالگوها، نقشه راه، FAQ

آناتومی یک پرامپت خوب برای عامل

۱. هدف

چه نتیجه‌ای می‌خواهی؟ (نه فقط «درستش کن»)

۲. Context

Spec، فایل‌ها، کد مشابه موجود، تیکت

۳. محدودیت‌ها

چه چیزی را تغییر نده، Non-goals، الگو

۴. تأیید

چطور بفهمیم درست است؟ تست، AC، build

۵. خروجی

Plan؟ PR؟ خلاصه؟ قالب مشخص

۶. حالت

اول Plan Mode، بعد اجرا

قاعده Anthropic: اگر diff را در یک جمله می‌توانی توصیف کنی، Plan لازم نیست؛ وگرنه اول Plan.

پرامپت بد در برابر پرامپت خوب

✗ بد

یه API برای پیگیری اقساط بنویس.

بدون Spec، بدون محدودیت، بدون معیار تأیید ← عامل حدس می‌زند.

✓ خوب

فیچر specs/001-installment-followup/spec.md را پیاده کن.

۱. اول با subagent بررسی کن InstallmentService فعلی چطور کار می‌کند.

۲. در Plan Mode فایل‌ها و ترتیب را پیشنهاد بده و صبر کن تا تأیید کنم.

۳. از dotnet-cqrs-feature Skill پیروی کن.

۴. به Domain و جدول‌های موجود دست نزن؛ ارسال پیامک خارج از محدوده است.

۵. هر معیار پذیرش ۱ تا ۴ باید یک unit test داشته باشد؛ dotnet test سبز.

۶. در پایان خلاصه تغییرات + نگاشت AC به تست بده.

ضدالگوها، اینها را نکنید

✗ پذیرش بدون خواندن

مالک کد شما باشید؛ PR عامل = همان review.

✗ یک چت بی پایان

context پر و آلوده می شود؛ برای هر تسک جلسه جدید.

✗ «درستش کن» بدون تست

عامل به خودش نمره می دهد.

✗ AGENTS.md هزار خطی

هر خط هزینه دائمی context دارد.

✗ اصلاح دستی خطای تکراری

به جایش Rule/Skill/تست اضافه کن (روی حلقه).

✗ Secret در پرامپت یا Rule

رمز و توکن هرگز در متن عامل.

نقشه راه پیشنهادی برای تیم ما

۳۰ روز اول

- AGENTS.md برای ۲ سرویس
- Rule ۲ (معماری، تست)
- آموزش Plan Mode
- اندازه‌گیری پایه: lead time، باگ

۶۰ روز

- Skill ۳ مشترک (.NET CQRS، migration، تست)
- اولین فیچر با Spec کامل
- subagent بازبین در PR
- Hook برای build/test

۹۰ روز

- SDD برای همه فیچرهای جدید
- Spec-anchored برای ماژول‌های اصلی
- کتابخانه Skill تیمی
- بازنگری داده‌ها و تنظیم خودمختاری

نقش‌ها: Champion (پیشگام) · Tech Lead (مالک Harness) · Manager (حمایت و اندازه‌گیری)

پرسش‌های پرتکرار (۱)

قانون کسب‌وکار Rule است یا Skill؟

معمولاً هیچ‌کدام: مستند دامنه + معیار پذیرش Spec. Rule فقط قرارداد کدنویسی.

آیا AI جای توسعه‌دهنده را می‌گیرد؟

نه؛ نقش را تغییر می‌دهد: طراحی، Spec، بازبینی، Harness. مالکیت و مسئولیت با انسان است.

Spec-first یا Spec-anchored؟

شروع با spec-first؛ برای مازول‌های مهم و پرتغییر spec-anchored.

BRD را داخل AGENTS.md بگذاریم؟

نه. BRD در docs/؛ خلاصه قابل تست در Spec.

پرسش‌های پرتکرار (۲)

توهم (Hallucination)؟

با context درست، تست قطعی، و بازبینی کاهش می‌یابد، حذف نمی‌شود.

امنیت کد شرکت؟

Privacy mode، عدم ارسال secret، بررسی سیاست داده هر ابزار، محدودیت دسترسی MCP و ترمینال.

کدام مدل؟

مدل قوی برای Plan و کار سخت؛ مدل سریع/ارزان برای کار تکراری. Harness مهم‌تر از مدل است.

جونیورها چطور یاد بگیرند؟

از عامل «توضیح» بخواهند، Plan را بخوانند، و کد را خودشان review کنند؛ نه فقط accept.

- Böckeler. Understanding Spec-Driven-Development: Kiro, spec-kit, and Tessel. martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html
- Böckeler. Context Engineering for Coding Agents. martinfowler.com/articles/exploring-gen-ai/context-engineering-coding-agents.html
- Böckeler. Harness engineering for coding agent users. martinfowler.com/articles/harness-engineering.html
- Kief Morris. Humans and Agents in Software Engineering Loops. martinfowler.com/articles/exploring-gen-ai/humans-and-agents.html
- Thoughtworks Podcast. What is spec-driven development? (Böckeler & Laura Tacho)
- Agent Skills specification. agentskills.io · github.com/agentskills/agentskills
- AGENTS.md. agents.md
- Cursor docs. Rules & Skills. cursor.com/docs
- Anthropic. Best practices for Claude Code. code.claude.com/docs/en/best-practices
- VS Code. Agent Skills. code.visualstudio.com/docs/agent-customization/agent-skills
- GitHub Spec Kit. github.com/github/spec-kit
- DORA 2025. State of AI-assisted Software Development. dora.dev/research/2025
- METR. Impact of Early-2025 AI on Experienced OSS Developer Productivity. metr.org



سپاس پرسش و پاسخ

همه فایل‌ها، نمونه‌ها و قالب‌ها روی گیت‌هاب:

github.com/mohammadrezatahmasebi/ai-agentic-meeting

/slides.pdf · handbook.pdf · samples/ · templates